Университет ИТМО, факультет программной инженерии и компьютерной техники
Двухнедельная отчётная работа по «Информатике»: аннотация к статье

Дата прошедшей лекции	Номер прошедшей лекции	Название статьи/главы книги/видеолекции	Дата публикации (не старше 2022 года)	Размер статьи (от 400 слов)	Дата сдачи
10.09.2025	1	Компьютеры на основе тернарной логики и перспективы их развития.	12.10.2022	~12500	24.09.2025
24.09.2025	2	Алгоритм сжатия информации с использованием нумерации перестановок.	2025	~1750	08.10.2025
08.10.2025	3	Enhanced Regular Expression as a DGL for Generation of Synthetic Big Data	2023	~6800	22.10.2025
22.10.2025	4	Language Server Protocol: An Introduction to the Protocol, its Use, and Adoption for Web Modeling Tools.	2023	~6600	05.11.2025
05.11.2025	5	The Programming Exercise Markup Language: Towards Reducing the Effort Needed to Use Automated Grading Tools	2023	~3800	19.11.2025
19.11.2025	6	A large-scale format compliance checker for TeX Font Metrics	2024	~1750	03.12.2025
03.12.2025	7	Using Large Language Models to Generate JUnit Tests: An Empirical Study	2024	~8400	17.12.2025

Выполнил(а) Шалабодов Я.Д., № группы *P3110*, оценка _____
Фамилия И.О. студента — не заполнять

Прямая полная ссылка на источник или сокращённая ссылка:
https://arxiv.org/pdf/2305.00418

Теги, ключевые слова или словосочетания:
Test generation, Large Language Models, JUnit, Java, unit testing, test smells, automated testing.

Перечень фактов, упомянутых в статье:

1. Современные недообученные LLM используются для генерации кода, но эффективность создания ими тестов для таких строго типизированных языков, как Java, до сих пор неизвестна.
2. Проведено эмпирическое исследование с использованием трёх LLM: Codex, GPT-3.5-Turbo, StarCoder, о том, как хорошо они генерируют тесты.
3. LLM показали хорошие результаты на простом наборе HumanEval (avg. ≈80%), но крайне низкие на EvoSuite SF110 (avg. <2%).
4. Сгенерированные тесты страдали от test smells: Duplicated Asserts and Empty Tests.
5. LLM часто “галлюцинирют”, создавая несуществующие в Java типы данных и методы, что приводит к ошибкам компиляции.
6. Полезно использовать минимальный контекст, содержащий MUT’s implementations при генерации тестов.
7. Такие модели, дообученные для задач, связанных с кодом, как Codex и StarCoder показали себя лучше, чем общая диалоговая модель GPT-3.5-Turbo, потому что лучше приспособлены к работе с кодом.
8. Несмотря на низкое качество, LLM могут быть полезны для Test Driven Development.

Позитивные следствия и/или достоинства описанной в статье технологии.

1. Те, кто практикует подход TDD могут значительно ускорить тестирование с помощью LLM.
2. Модели LLM способны генерировать тесты, даже когда есть только описание метода без конкретной структуры.
3. Генерируются тесты, которые выходят за рамки очевидных задач, что позволяет более детально протестировать код.

Негативные следствия и/или недостатки описанной в статье технологии.

1. Пока что низкая надёжность и высокий уровень “галлюцинаций”, что мешает нормальному тестированию.
2. В сгенерированных тестах часто содержатся test smells, что делает их понимание в разы труднее, усложняет документацию.
3. Производительность плоха для проектов с открытым исходным кодом из Evosuite “based on coverage”.